

基于安全协议代码的形式化辅助建模研究^{*}

葛 艺, 黄文超[†], 熊 焰

(中国科学技术大学 计算机科学与技术学院, 合肥 230000)

摘 要: 随着安全协议形式化分析技术的不断发展,利用工具自动验证虽已得到实现,但建模环节仍需依赖专业人员手工建模,难度大且成本高,限制了此技术的进一步推广。为了提高建模的自动化程度,提出了依据安全协议代码进行形式化模型辅助生成的方案。首先,使用污点分析获取协议的通信流程,并且记录密码学原语操作;然后,根据通信流程之间的序列关系构建协议通信状态机;最终,根据目前主流的安全协议形式化分析工具 Tamarin 的模型语法生成形式化模型。实验结果表明,此方案可以生成形式化模型中的关键部分,提高了形式化建模的自动化程度,为形式化分析技术的推广作出贡献。

关键词: 形式化验证; 形式化建模; 协议代码; 污点分析; Tamarin

中图分类号: TP309

文献标志码: A

文章编号: 1001-3695(2023)04-037-1189-05

doi:10.19734/j.issn.1001-3695.2022.08.0446

Research on computer-aided formal modeling for security protocol code

Ge Yi, Huang Wenchao[†], Xiong Yan

(School of Computer Science & Technology, University of Science & Technology of China, Hefei 230000, China)

Abstract: With the development of formal analysis technology of security protocols, automatic verification using tools has been realized, but modeling still needs to rely on manual modeling by professionals, which is difficult and costly and limits the further spread of formal analysis technology. In order to improve the automation of formal modeling, this paper proposed a scheme of formal assistant modeling based on security protocol code. Firstly, the method used taint analysis to obtain communication processes and record the cryptographic primitive operations. Then, it constructed the protocol communication state machine according to the sequential relationship between communication processes. Finally, it generated a formal model according to the syntax of Tamarin, a mainstream formal analysis tool for security protocols. Experimental results show that this scheme can generate the key parts of the formal model and improves the degree of automation of the formal modeling, which contributes to the spread of formal analysis technology.

Key words: formal verification; formal modeling; protocol code; taint analysis; Tamarin

0 引言

安全协议是以密码学为基础的信息交换协议,为计算机系统中的信息安全提供了基础保障。然而,安全协议的运行环境通常是不安全的,协议的设计一旦存在缺陷,就会造成不可挽回的后果。为了避免这种情况的发生,需要尽早地对协议进行安全性验证。安全性验证包括人工审查以及形式化验证,人工审查的效率较低且准确性不高,相比之下,形式化验证通过更加正规、严谨的方法对安全协议进行验证,找出协议中存在的漏洞,对协议的设计起到重要的指导作用。早在1996年,文献[1]就利用形式化分析发现了NSPK协议的中间人攻击。随后,研究者们开发并使用形式化分析工具分析了大量安全协议,比如AVISPA工具针对IKEv2协议、H.530协议以及ISO-PK协议进行了形式化分析并且发现了相关攻击^[2~4];Scyther对IKE协议族进行了分析,发现了IKEv1和IKEv2协议的并行攻击^[5]。利用形式化验证工具对安全协议进行分析是一种准确高效的方式,以相应的形式化模型为输入,通过工具进行逻辑推理,最后输出验证结果。近十年来,安全协议的形式化验证研究取得了显著进展,目前主流的形式化验证工具(如Proverif^[6]、AVISPA^[7]、Tamarin-prover^[8]、Scyther^[9]等)可以在几秒钟

内自动验证Kerberos V或TLS握手等经典协议模型的机密性和身份验证属性,对无数个会话进行验证,其中Tamarin-prover拥有较丰富的表达能力,常被用于安全协议分析。

安全协议的形式化验证包含建模和验证两个主要步骤,建模阶段构建形式化模型,在验证阶段对其进行验证得到最终的验证结果。目前,已经有研究实现了验证步骤的全自动化^[10],但是在建模阶段仍然是以手工建模为主。形式化建模以协议描述为依据提取出安全协议描述,协议描述可以是协议规范或者协议代码,人工建模通常选取可读性更高的协议规范作为依据,复杂安全协议的规范说明长达上百页,包括严格的协议机制说明、隐式的安全保障以及介绍性文字,这要求建模人员拥有较高的专业技能,是一项非常繁重的工作。此外,由于各形式化验证工具对模型要求的差异,形式化模型的通用性往往很低,限制了形式化分析技术的推广。除了完全依赖人工建模以外,还有一部分辅助建模的工作,Cremers等人^[11]利用Tamarin对TLS1.3协议进行了形式化分析,在建模时不是将协议规范直接转换成Tamarin协议模型,而是利用m4预处理器定义了大量的宏,可以覆盖协议规范的大部分,同时在语法上接近Tamarin模型语言,以这种方式构造模型可以实现与协议规范的直接比较,同时可以根据需求灵活修改。尽管这在一定程度

收稿日期: 2022-08-19; 修回日期: 2022-10-31 基金项目: 国家自然科学基金面上项目(61972369); 国家自然科学基金青年项目(62102385); 安徽省自然科学基金资助项目(2108085QF262)

作者简介: 葛艺(1998-),女,安徽合肥人,硕士,主要研究方向为信息安全; 黄文超(1982-),男(通信作者),湖北宜昌人,副教授,硕士,主要研究方向为网络安全、形式化验证等(huangwc@ustc.edu.cn); 熊焰(1960-),男,安徽宿松人,教授,博士,主要研究方向为计算机网络与信息安全、移动计算等。

上提高了模型的灵活性,但是仍然偏向手工建模,需要大量专家的参与,没有从根本上解决形式化建模的痛点。

与协议规范不同,协议代码作为安全协议的实现方式,往往由高级程序语言编写而成,不但有着严格的语法并且包含了代码实现的底层操作以及逻辑层面的协议流程,在严格遵循协议规范的情况之下,实现了协议流程中的所有步骤,对其进行自动的形式化建模可以辅助安全协议的形式化验证分析,为研究人员提供了另一种建模参考和思路,减轻了人工建模的负担,降低了形式化验证技术的门槛。Stephens 等人^[12]通过将智能合约以及要满足的安全属性转换成 Buchi 自动机来进行验证,然而智能合约的代码比较简单,相较于用复杂编程语言实现的安全协议代码,对其进行自动转换较好实现。对于复杂的编程语言,文献^[13]将安全协议的 Web 实现转换为应用 Pi 演算模型,半自动地完成了从协议代码到验证模型的转换,但是此工作仅适用于 Python 编写的协议实现代码。文献^[14]则是将函数调用抽象为函数调用时序图,并基于静态执行下的函数调用序列,给出了 C 程序中函数调用在 Pi 演算模型中的抽象方法,但是所选取的形式化验证工具限制了工具适用的范围。除此之外,还有一部分工作直接对安全协议代码进行建模,将程序代码转换为计算模型^[15]、Petri 网^[16]模型等,尽管这些工作可以实现从代码到模型的自动转换,但是只能够针对代码的底层漏洞进行检测,并不能够检测到代码里的协议逻辑漏洞。

从协议代码到形式化模型的转换本质上是对协议代码进行抽象的过程,需要采用一种中间形式来完成这样一种转换,而代码的运行过程与形式化模型的推理过程都可以被视为状态机的转换过程,所以本文采用了状态机作为中间转换形式,由于此状态机包含的内容是协议通信的过程,称之为通信状态机。从协议代码到通信状态机的转换实际上是对代码进行信息提取,提取出与建模相关的关键信息,即协议的通信流程,采用程序分析中的数据流跟踪,而从通信状态机到形式化模型的转换需要根据具体的形式化验证工具的模型语言来制定具体的转换规则。综上,本文提出了一种基于污点分析的模型生成方法,整体框架如图 1 所示。首先,利用污点分析方法提取协议代码中的通信流程信息,并且记录消息经过的密码学原语操作,然后根据协议通信流程之间的序列关系构建通信状态机,最后依据形式化模型的语法,将状态机的状态转移边转换为对应的形式化模型语句,自动化地完成形式化建模,减轻人工负担,降低学习成本,推进形式化分析技术的发展。

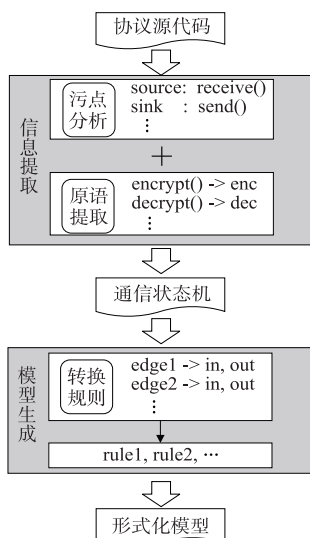


图1 形式化辅助建模方案框架

Fig. 1 Framework of computer-aided formal modeling

1 基本理论

1.1 污点分析

污点分析是程序数据流分析的一个典型应用^[17]。当存储在 x 中的数据被传入 y 时,就产生了从 x 流向 y 的数据流,记做 $x \rightarrow y$ ^[1]。在污点分析中,当一个变量 v 的来源是未知的或者不被信任的,它将带有污点标记,称为被污染的 (tainted) 数据,如果对这个变量进行操作 op 获得一个返回值 rt ,根据这个返回值是否也会变为被污染的数据,可以将操作 (op) 分为两类。如果 rt 经过 op 后也变为了污点数据,那么 op 被称为传播操作,反之, op 称为过滤操作。其中,传播操作具有传递性,也就是说,如果带有污点标记的变量 x 通过操作 op 得到 y , y 又通过操作 op 得到 z ,则产生了一条污点传播路径 $x \rightarrow z$ 。污染分析的 API 由以下四组操作组合而成,直观过程如图 2 所示。

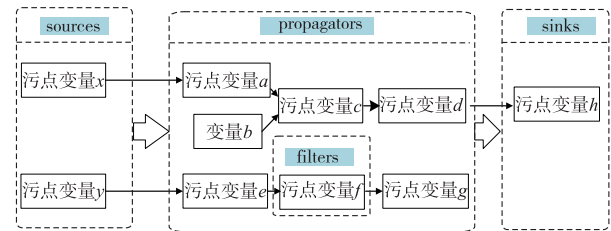


图2 污点分析过程

Fig. 2 Process of taint analysis

- a) sources, 即污点源,代表可以生成污染数据的操作集。
- b) propagators, 即传播操作,表示传播或转换污染数据的操作集。编程语言中的复制操作,比如赋值和参数传递,都被认为是传播操作,如果存在指针或者引用,还要考虑别名现象。
- c) filters, 即过滤操作,是从污染数据生成安全数据的操作集。
- d) sinks, 即汇聚点,是一组用于判断是否使用了污点数据的操作集,此类操作是需要重点关注的。

1.2 Tamarin 模型

Tamarin 基于多重集合改写规则描述协议流程,利用一阶逻辑量化协议消息和时间节点,从而实现对协议属性的描述,支持具有复杂控制流的协议或者具有非单调易变全局状态的协议,支持 eCK 等强安全模型,支持多种代数性质:加/解密运算、运算结合律、Diffie-Hellman 幂运算以及双线性对运算,它是目前主流的针对安全协议的形式化验证工具,被广泛地应用于各类安全协议的分析。Tamarin 的输入即为 Tamarin 模型,被定义在 `spthy` 文件中,通过重写规则指定协议中不同角色的行为。这些重写规则定义了有标记的转换系统,它的状态包括对手的知识、网络消息、协议状态等。Tamarin 使用方程式理论来支持常见的密码原语,如加密和哈希,并且使用跟踪属性对安全属性进行建模。下面对 Tamarin 模型中两个重要的组成部分进行简单的介绍:

- a) 规则 (rules)。一条重写规则包含规则的名字以及三个部分,每一个部分都是事实 (facts) 的集合,分别称为前提 (premise)、动作 (action) 和结论 (result)。规则的名称仅用于引用特定的规则。它们没有特定的含义,可以任意选择,只要每个规则都有一个唯一的名称即可,如果一个规则没有被 action 事实标记,那么箭头符号 `--[ ]->` 可以缩写为 `-->`。转移系统的初始状态为空的多重集合,规则定义了系统如何向新状态进行转移。如果一个规则可以实例化 (规则左侧包含在当前状态下),则可以将该规则应用于该状态。在这种情况下,左侧的事实被从状态中移除,并被右侧实例化所取代。比如,针对以下的这条规则,名字为 `rule_example`,前提为 `[ fact1, fact2 ]`,

当前提得到满足时,应用这条规则可以得到[fact3,fact4]。

```
rule rule_example;
[ fact1,fact2 ] --[ action fact ] -> [ fact3,fact4 ]
```

b) 事实(facts)。事实以 $F(t_1, \dots, t_n)$ 的形式表示,其中 F 是事实的名字, t_1 到 t_n 为参数,参数的个数是固定的,为 n 。在 Tamarin 中,内置了三个具有重要意义的现实:

(a) in 事实。用于建模在 Dolev-Yao 攻击者控制的不可信网络上接收消息的行为,并且只能发生在重写规则的左侧。

(b) out 事实。用于建模在 Dolev-Yao 攻击者控制的不可信网络上发送消息的行为,并且只能发生在重写规则的右侧。

(c) Fr 事实。当需要生成新变量时,必须使用这个事实,并且只能发生在重写规则的左侧,其中它的参数是新的项,可以用于建模产生随机值的行为。

2 基于污点分析的信息提取

信息提取模块包含消息传递流程提取以及密码学原语提取两个部分。消息传递流程即协议参与方之间收发消息的流程,是形式化模型中的主要组成部分,消息传递流程提取作为自动化建模的第一步,为构建形式化模型整体框架奠定了基础。密码学原语则为不需要展开分析的密码学操作函数,在形式化模型中用于描述消息所经过的密码学操作,进行密码学原语提取是为构建更加精细的形式化模型。本工作通过污点分析跟踪协议代码中的收发消息的 API 来获取消息传递流程,与此同时,在跟踪收发消息流程时,记录消息经过的密码学原语。在后续建模时,依据信息提取模块获得的信息对安全协议流程以及消息经过的密码学原语进行描述。

2.1 消息传递流程提取

消息传递流程是指协议参与方之间传递消息的步骤,包括发送方、接收方以及参与方之间传递的消息,多条消息传递流程共同组成了安全协议的通信流程。在用形式化模型对安全协议进行描述时,主体框架即为消息传递流程,因此,首先需要通过分析协议代码获取消息传递流程。本文方案利用污点分析技术提取安全协议代码中的收发消息数据流。首先,将收发消息数据流提取映射到污点分析的情形之下,将接收消息的 API 作为 source,发送消息的 API 作为 sink,可以用如图 3 所示的有限状态机来描述,对应的五元组为 (Q, P, δ, q_0, F) , 其中:

$Q = \{q_0, q_1, q_2\}$,
 $P = \{\text{source}(), \text{sink}()\}$,
 δ 为转换函数, $\delta(q_0, \text{source}()) = q_1$, $\delta(q_1, \text{sink}()) = q_2$,
 $q_0 = \{q_0\}$,
 $F = \{q_2\}$ 。

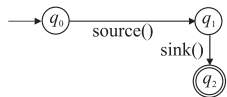


图3 有限状态机
Fig. 3 Finite-state machine

获取的一条污点分析路径,代表着一次消息收发过程,具体包含了消息来源方、消息接收方以及传递的消息变量,此处将这些信息记录下来以备后续使用。

2.2 密码学原语提取

消息传递流程中的消息一般都会经过一系列的密码学操作,即密码学原语,为了在形式化模型中对其进行更加精准的描述,在跟踪收发消息数据流的同时,记录消息经过的一系列密码学原语。

在协议代码中,密码学操作往往由函数来实现,在密码学函数中包含了代码实现的具体细节,例如,下面的函数表示加密操作,包含了加密算法的具体实施以及错误判断。由于在构

建形式化模型时采取完美密码理论,可以忽略密码学操作的具体实现,将密码学函数作为黑盒处理,只关注函数的密码学功能,而不关注具体的密码学算法。所以,以上的函数直接抽象为加密原语 enc。

```
int encrypt(void * buffer, int buffer_len, char * IV, char * key, int key_len)
{
    MCRYPT td = mcrypt_module_open("rijndael-128", NULL, "cbc", NULL);
    int blocksize = mcrypt_enc_get_block_size(td);
    if(buffer_len % blocksize != 0) { return 1; }
    mcrypt_generic_init(td, key, key_len, IV);
    mcrypt_generic(td, buffer, buffer_len);
    mcrypt_generic_deinit(td);
    mcrypt_module_close(td);
    return 0;
}
```

在本文中关注两类密码学原语,即加密原语和解密原语。

定义1 密码学原语序列 $\text{CryOpt}(op_1, op_2, \dots)$, op 为解密原语或者加密原语,加密原语和解密原语分别用 enc 和 dec 表示。

类似 source 和 sink 的设置,在污点分析开始之前,根据代码预先设置好加/解密原语的 API,在进行污点分析的同时,记录经过的密码学操作,组成密码学原语序列 CryOpt 。

3 协议通信状态机构建

通过污点分析获取到消息传递流程之后,为了表示这些消息传递流程之间的序列关系,构建了安全协议的通信状态机,作为协议源代码向形式化模型转换的中间表示形式。针对每一条消息传递流程,发送的消息往往取决于接收的消息,根据这个特点,本文方案采取米利状态机(Mealy machine)的结构。

米利状态机是基于当前状态和输入生成输出的有限状态自动机,它的状态图的每个转移边包括输入和输出,一个 Mealy machine 由一个六元组 $(S, I, O, \delta, X, s_0)$ 表示,其中 S 是一个状态的有限集, I 是输入的有限集, O 是输出的有限集, δ 是状态的转换关系集合 $S \times I \rightarrow S$, X 是输出的转换关系集合 $S \times I \rightarrow O$, s_0 是初始状态,且 $s_0 \in S$; δ 中的一个转换关系 $s_k \times i_k \rightarrow s_{k+1}$ 表示从当前状态 s_k 出发,接收输入 i_k 后,状态转换为 s_{k+1} ; X 中的一个转换关系 $s_k \times i_k \rightarrow o_k$ 表示从当前状态 s_k 出发,接收输入 i_k 后,输出为 o_k ; 其中, $s_k \in S, s_{k+1} \in S, i_k \in I, o_k \in O$ 。Mealy machine 的输出取决于当前状态和当前输入,从初始状态出发,给定一个输入序列,状态转换序列和输出序列也是确定的。因此,当初始状态给定的情况下,可以用 Mealy machine 中从初始状态 s_0 出发到某一目标状态 s_i 的所有输入序列集合来表示状态的信息,这是将协议实现的 Mealy machine 抽象信息用于指导形式化建模的理论基础。

基于已经得到的污点分析路径,本文设计了协议的状态机构建算法。每个协议参与方 R 都拥有唯一标识符 ID,根据污点路径出现的先后顺序为每个协议参与方 R 生成通信流程 $R.\text{Seq}$,每一条污点路径作为通信流程序列中的一个元素,表示一次收发消息操作,包括消息来源方、消息接收方、消息以及消息经过的密码学原语,将通信流程序列定义如下:

定义2 每一个协议参与方 R 包含唯一标识符 ID 和通信流程队列 Seq ,其中,通信流程序列 $R.\text{Seq}$, $R.\text{Seq}[i]$ 表示第 i 条污点路径包含的信息,可以展开为 $(\text{from}, \text{to}, \text{mes}, \text{dec}, \text{enc})$,其中 from 表示消息来源方, to 表示消息接收方, mes 表示发送的消息变量, dec 表示解密原语的次数, enc 表示加密原语的次数。

协议通信状态机构建的具体流程如算法1所示。首先,指

定协议发起方 S 并且配置初始通信信息 V_0 , 包括初始消息的接收方以及经过的密码学原语, 将 V_0 作为状态机的初始状态节点。然后, 到当前参与方 $curNode$ 的消息接收方 $curRule$ 的通信流程序列中获取头节点, 作为 $curNode$ 节点对应状态的下一个状态, 直到所有的参与方的通信流程序列皆被遍历, 算法结束, 返回状态机 M 。

算法1 协议状态机构建算法

输入: 协议参与方集合 $Role$; 协议发起方 S 以及初始通信信息 V_0 (to, mes, dec, enc)。

输出: 协议状态机 M 。

a) if ($Role[i].ID.equal(S)$) 在 $Role[i].Seq$ 队列头部加上 V_0 ;
//初始配置

b) 将 V_0 作为 M 的起始节点;

c) $curNode = V_0; curRule = S$;

d) 删除 $curRule.Seq$ 队头;

e) $curRule = curNode.To$;

f) $nextNode = curRule.Seq.front()$;

g) 创建从 $curNode$ 到 $nextNode$ 的边;

h) 重复步骤 c) ~ g), 直到每个协议参与方的通信流程队列 Seq 都为空。

i) 返回 M 。

4 形式化模型生成

本文所选取的形式化验证工具 Tamarin 对应的形式化模型被定义在 $spty$ 文件中, 包含几个组成部分: 协议流程模型、攻击者模型以及安全属性。由于本文只关注协议本身的建模, 所以攻击者模型以及安全属性不在考虑范围内。协议流程模型由多条规则 (rules) 组成, 每一条规则拥有唯一的规则名称, 包含前提、动作和结论三个部分, 每一个部分都是事实 (facts) 的集合, 下面详细描述从通信状态机到具体规则的转换方式。

一次状态转移包含一对收发消息操作, 如图4所示, 状态 s_1 接收消息 m , 并且发送解密后的消息 $dec(m, k)$, 状态变为 s_2 。

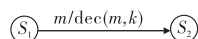


图4 通信状态转移示例

Fig.4 Example of communication states transition

转换前的状态记录在规则的前提部分, 转换后的状态记录在规则的结果部分, env 表示当前状态下已知的各类消息。上述例子转换为 Tamarin 模型的规则为

```

rule s1_s2:
  [ s1(env), in(m) ]
  --[ Action_s1_s2 ]->
  [ s2(env), out(dec(m,k)) ]
  
```

通信状态机中从初始状态到终止状态的一条路径对应的就是协议的一次运行流程, 其中的每一步状态转移都可以生成一条 Tamarin 规则, 对状态转移路径进行一次遍历, 最终实现从 Mealy machine 中提供的协议状态信息生成形式化模型规则。

5 实验案例与分析

5.1 实验设计与实现

目前已有的利用协议代码自动建模的方案并不多, 并且都关注于形式化验证的结果^[11,15], 并没有对生成的形式化模型做过多的分析, 而本文关注于模型生成的过程与质量; 另外, 针对同一个协议最终构建的形式化模型并不是唯一的, 有多种描述方式。基于以上原因, 本文的实验结果难以与其他自动建模方案进行横向或纵向的对比。由于缺少同类方法的比较, 本文设计了五个指标量化了模型的生成效果, 也将此作为后续工作改进的基准。在分析实验结果时, 为了验证污点分析提取协议流程信息的准确性, 需要对所有的污点路径进行人工分析, 将

每一条污点路径与协议流程进行匹配, 对匹配结果进行分析。其次, 为了验证整体方案的有效性, 采取将生成的模型与手工建模的模型进行对比的方式, 从以下几个维度进行分析, 其中手工建模的模型采用的是 Tamarin 官方案例中提供的形式化模型:

a) 规则个数 (NR), 一个形式化模型文件中所有规则 (rules) 的个数。

b) in 事实个数 (NI), 所有 rules 中 in 事实的个数。

c) out 事实个数 (NO), 所有 rules 中 out 事实的个数。

d) 参数个数 (NP), 所有事实中包含的参数的个数。

e) 密码学原语个数 (NC), 所有事实中包含的加/解密操作的个数。

以上五个指数越接近, 表示生成的模型与手工建模的模型相似性越高, 因此通过计算两者的百分比可以直观地表示出模型的相似性, 由于官方提供的手工建模模型已经被实际检验过, 可以认为相似性越高, 方案的有效性就越高。

本文选取的安全协议代码为 C 语言实现, 系统内置了代表接收和发送消息的函数作为 source 和 sink, 另外用户也可以根据需求自行定义, source 和 sink 作为配置文件与协议 C 代码共同作为系统的输入, 在进行污点分析时采用 clang-4 作为后端, 利用 clang static analyzer 提供的接口实现用户可配置的污点分析, 然后将结果作为算法1的输入得到通信状态机, 最后根据模型生成规则得到形式化模型。

实现过程可分为以下几个步骤: a) 将 source 和 sink 作为配置文件, 与协议 C 代码输入到系统中; b) 污点分析器读取配置文件, 根据 source 和 sink 分析协议 C 代码, 返回污点分析路径; c) 指定协议参与方集合 role, 协议发起方 S 以及初始通信信息 V_0 , 根据算法1计算通信状态机; d) 根据第4章的转换规则, 将通信状态机转换为 Tamarin 模型。

5.2 案例分析

Needham-Schroeder 协议 (NS 协议) 是一个基于对称加密算法的协议, 它要求有可信任的第三方 KDC 参与, 采用 challenge/response 的方式, 使得 A、B 互相认证对方的身份。A 是协议的发起者, B 是协议的响应者, K 是认证服务器, 是 A 生成的随机数, Nb 是 B 生成的随机数, Kab 是 S 生成的会话密钥, Kas 是 A 和 S 的共享密钥, Kbs 是 B 和 S 的共享密钥。在完成上述协议后, A 和 B 就能确信他们之间拥有一个仅为他们和认证服务器知道的会话密钥 Kab, 协议流程如下:

```

a) A->K: A,B,Na
b) K->A: { Na,B,Kab, { Kab,A } Kbs } Kas
c) A->B: { Kab,A } Kbs
d) B->A: { Nb } Kab
e) A->B: { succ(Nb) } Kab
  
```

在进行污点分析时, 如图5所示, KDC.c 文件中, recvfrom 函数将污点标记赋给 request 结构体, 然后 request 通过字符串拷贝操作将污点标记传播给 reply 中的成员变量 receiverHostname, 在汇聚函数 sendto 中判断第一个参数是否被污染, 而第一个参数为 reply 结构体对象。

```

//get tainted
len = recvfrom(servSocket, (void *) request, sizeof(struct kdc_request_s),
0, (struct sockaddr *) &remoteAddr, (socklen_t *) &addrlen);
...
// propagate
strcpy(reply->receiverHostname, request->receiverHostname);
...
// sink
len = sendto(servSocket, (void *) reply, sizeof(struct kdc_reply_s),
0, (const struct sockaddr *) &remoteAddr, (socklen_t) addrlen);
  
```

图5 KDC.c 局部代码

Fig.5 Part code of KDC.c

对于用于挖掘漏洞的污点分析而言, 为了避免假阳性过高

的问题,这种情况下并不会产生污点数据泄露报告,但是为了达到本文协议流程提取的目的,需要提高污点传播的能力,当结构体成员变量被污染时认为结构体对象也是被污染的,从而产生污点数据泄露报告,对应一条协议流程路径,如图6所示。

```
len = sendto(servSocket, (void *)reply, sizeof(struct kdc_reply_s), 0,
(const struct sockaddr*)&remoteAddr, (socklen_t) addrlen);
```

5 Untrusted data '(void *)reply' is passed to this sink. No filter found since it got tainted, make sure to sanitize before passing to it

图6 KDC.c 污点分析结果
Fig. 6 Taint analysis results of KDC.c

将污点分析结果转换为对应的状态转移如图7所示。

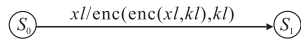


图7 KDC 状态转移图
Fig. 7 States transition of KDC

然后,依据 Tamarin 的语法规则,构建出的形式化规则为

```
rule s0_s1 :
[ S0( env ), in( x1 ), ]
--[ action( s0_s1 ) ] ->
[ S1( env ), out( enc( enc( x1, k1 ) ), k1 ) ]
```

其他规则的生成也采取类似的方法,规则生成的顺序与通信状态机的状态转移顺序一致,下一条规则的 in 事实中包含的参数与上一条规则中 out 事实的参数一致,表示参与方之间的消息收发顺序。

5.3 实验结果与分析

通过以上的案例分析,得到了最终生成的形式化模型,对其进行进一步分析,目的在于验证污点分析在此方案中的有效性以及评估生成的形式化模型的质量。首先,对污点分析提取协议流程信息的正确性进行验证,结果如表1所示。污点分析一共返回了四条污点路径,经过比对,包含了协议流程中除发起步骤之外的所有步骤,对于发起步骤,由于其并没有接收消息的前提,也就没有接收消息的 API 可以作为污点源,所以无法通过污点分析得到,在后续的状态机构建算法中通过事先指定协议发起方以及初始通信信息来补全这里的缺失。因此,污点分析对于本案例的有效性得到证实。

表1 污点路径分析结果
Tab. 1 Result of taint paths analysis

协议方	污点路径数目	文件名	对应的协议流程
A	2	Alice. c	A -> B: { Kab, A } Kbs A -> B: { succ(Nb) } Kab
K	1	KDC. c	K -> A: { Na, B, Kab, { Kab, A } Kbs } Kas
B	1	Bob. c	B -> A: { Nb } Kab

其次,将生成的 NS 协议的形式化模型与 Tamarin 中手工建模的模型进行对比,指标统计结果如表2所示。

表2 模型指标统计结果
Tab. 2 Result of model indicators statistics

模型	NR	NI	NO	NP	NC
人工建模模型	8	5	6	94	8
辅助建模模型	6	5	5	34	7

将生成的模型指标除以手工建模的指标得到如图8所示的柱状图。从实验结果可以看出,本文方案可以生成大多数的规则,经过分析,这些规则都与协议流程主体有关,这与污点分析提取协议流程的目的之一致,其余两条无法生成的规则是对攻击者能力的描述以及初始化参数的设置,这些都不包含在协议代码中,因此需要依赖协议代码以外的信息完成建模。由于目前方法难以分析协议代码中具体参数的含义,还无法对消息参数进行更细致的建模,在参数个数上与手工建模的模型差距较大。此外,通过分析代码可以对加/解密操作进行较为准确的建模,但是由于手工建模时对参数的建模更加细致并且采取了

约简措施,所以在密码学原语个数的指标上有所差异。实验结果表明,通过本文方案生成的形式化模型与手工建模的模型有较高的相似度,可以对安全协议的主要流程进行描述,但在提高自动化程度的同时损失了一些建模中的精确度,这需要在后续的工作中继续改进。

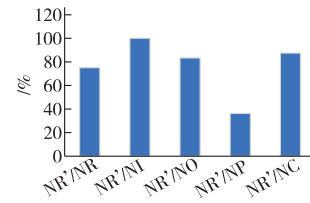


图8 模型指标占比
Fig. 8 Proportion chart of model indicators

6 结束语

形式化方法在安全协议验证中发挥着越来越重要的作用,自动化程度关系着形式化方法的进一步推广,针对形式化建模自动化程度低的问题,本文提出一种依据安全协议代码自动生成形式化模型的方法。使用污点分析方法以及状态机构建算法,获得了协议流程中的关键信息,构造出了符合 Tamarin 验证工具规范的形式化模型。对 NS 协议进行了自动模型生成实验,结果表明,此方法可以生成与手工建模的模型相似的形式化模型,但是精确度仍然达不到手工建模的程度。下一步可以对消息参数进行提取,建立消息之间的绑定关系,提高模型的精确度,同时,可以将此方法应用到更加复杂的安全协议代码之上,提高此方法的扩展性,从而进一步实现安全协议的形式化自动建模。本文探讨的是安全协议的形式化自动建模方案,实现全自动建模是最高目标,目前提出的方案可以实现辅助建模,在一定程度上减轻了人工建模的压力,降低了形式化验证的门槛。

参考文献:

- [1] Lowe G. Breaking and fixing the Needham-Schroeder public-key protocol using FDR[C]//Proc of International Workshop on Tools and Algorithms for the Construction and Analysis of Systems. Berlin: Springer, 1996: 147-166.
- [2] Ziauddin S, Martin B. Formal analysis of ISO/IEC 9798-2 authentication standard using avispa[C]//Proc of the 8th Asia Joint Conference on Information Security. Piscataway, NJ: IEEE Press, 2013: 108-114.
- [3] Islam S. Security analysis of LMAP using AVISPA[J]. International Journal of Security and Networks, 2014, 9(1): 30-39.
- [4] Vigano L. Automated security protocol analysis with the AVISPA tool[J]. Electronic Notes in Theoretical Computer Science, 2006, 155: 61-86.
- [5] Cremers C. Key exchange in ipsec revisited; formal analysis of IKEV1 and IKEV2[C]//Proc of European Symposium on Research in Computer Security. Berlin: Springer, 2011: 315-334.
- [6] Blanchet B, Cheval V, Cortier V. Proverif with lemmas, induction, fast subsumption, and much more[C]//Proc of Symposium on Security and Privacy. Piscataway, NJ: IEEE Press, 2022: 69-86.
- [7] Sharma S, Kaushik B, Rahmani M K I, et al. Cryptographic solution-based secure elliptic curve cryptography enabled radio frequency identification mutual authentication protocol for Internet of Vehicles[J]. IEEE Access, 2021, 9: 147114-147128.
- [8] Basin D, Cremers C, Dreier J, et al. Tamarin: verification of large-scale, real-world, cryptographic protocols[J]. IEEE Security & Privacy, 2022, 20(3): 24-32.
- [9] Dewangan N K, Chandrakar P. Patient-centric token-based healthcare blockchain implementation using secure Internet of medical things[J/OL]. IEEE Trans on Computational Social Systems. 2022. <http://doi.org/10.1109/TCSS.2022.3194872>. (下转第1202页)

- nal on Selected Areas in Communications, 2019, 37(3): 683-697.
- [12] Hecker C, Nance K L, Hay B. Dynamic honeypot construction[C]//Proc of the 10th Colloquium for Information Systems Security Education. 2006.
- [13] 陈启璋, 林国恩, 李建彬. 一种基于动态蜜罐和实时仿真的蜜网设计[J]. 微计算机信息, 2006, 22(12-3): 28-30. (Chen Qizhang, Lin Guo'en, Li Jianbin. Honeynet design based on dynamic honeypot and real-time emulation[J]. Microcomputer Information, 2006, 22(12-3): 28-30.)
- [14] 石乐义, 李捷, 刘昕, 等. 基于动态阵列蜜罐的协同网络防御策略研究[J]. 通信学报, 2012, 33(11): 159-164. (Shi Leyi, Li Jie, Liu Xin, et al. Research on dynamic array honeypot for collaborative network defense strategy[J]. Journal on Communications, 2012, 33(11): 159-164.)
- [15] 李之棠, 徐晓丹. 动态蜜罐技术分析与设计[J]. 华中科技大学学报: 自然科学版, 2005, 33(2): 86-88, 102. (Li Zhitang, Xu Xiaodan. The analysis of dynamic honeypot and its design[J]. Journal of Huazhong University of Science and Technology: Natural Science Edition, 2005, 33(2): 86-88, 102.)
- [16] 贾召鹏, 方滨兴, 崔翔, 等. ArkHoney: 基于协同机制的 Web 蜜罐[J]. 计算机学报, 2018, 41(2): 413-425. (Jia Zhaopeng, Fang Binxing, Cui Xiang, et al. ArkHoney: a Web honeypot based on collaborative mechanisms[J]. Chinese Journal of Computers, 2018, 41(2): 413-425.)
- [17] 郭江兴. 网络空间内生安全: 拟态防御与广义鲁棒控制(上册)[M]. 北京: 科学出版社, 2020: 190-249. (Wu Jiangxing. Endogenous security in cyberspace: mimic defense and generalized robust control (Volume I)[M]. Beijing: Science Press, 2020: 190-249.)
- [18] 郭江兴. 网络空间内生安全发展范式[J]. 中国科学: 信息科学, 2022, 52(2): 189-204. (Wu Jiangxing. Development paradigms of cyberspace endogenous safety and security[J]. Scientia Sinica: Informationis, 2022, 52(2): 189-204.)
- [19] 徐恪, 付松涛, 李琦, 等. 互联网内生安全体系结构研究进展[J]. 计算机学报, 2021, 44(11): 2149-2172. (Xu Ke, Fu Songtao, Li Qi, et al. The research progress on intrinsic internet security architecture[J]. Chinese Journal of Computers, 2021, 44(11): 2149-2172.)
- [20] 郭江兴. 网络空间拟态防御研究[J]. 信息安全学报, 2016, 1(4): 1-10. (Wu Jiangxin. Research on cyber mimic defense[J]. Journal of Cyber Security, 2016, 1(4): 1-10.)
- [21] 郭江兴. 网络空间拟态安全防御[J]. 保密科学技术, 2014(10): 4-9. (Wu Jiangxing. Cyberspace mimic security defense[J]. Secrecy Science and Technology, 2014(10): 4-9.)
- [22] 普黎明, 卫红权, 李星, 等. 面向云应用的拟态云服务架构[J]. 网络与信息安全学报, 2021, 7(1): 101-112. (Pu Liming, Wei Hongquan, Li Xing, et al. Mimic cloud service architecture for cloud applications[J]. Chinese Journal of Network and Information Security, 2021, 7(1): 101-112.)
- [23] 张杰鑫, 庞建民, 张铮. 拟态构造的 Web 服务器异构性量化方法[J]. 软件学报, 2020, 31(2): 564-577. (Zhang Jiexin, Pang Jianmin, Zhang Zheng. Quantification method for heterogeneity on Web server with mimic construction[J]. Journal of Software, 2020, 31(2): 564-577.)
- [24] 宋克, 刘勤让, 魏帅, 等. 基于拟态防御的以太网交换机内生安全体系结构[J]. 通信学报, 2020, 41(5): 18-26. (Song Ke, Liu Qingrang, Wei Shuai, et al. Endogenous security architecture of Ethernet switch based on mimic defense[J]. Journal on Communications, 2020, 41(5): 18-26.)
- [25] 马海龙, 伊鹏, 江逸茗, 等. 基于动态异构冗余机制的路由器拟态防御体系结构[J]. 信息安全学报, 2017, 2(1): 29-42. (Ma Hailong, Yi Peng, Jiang Yiming, et al. Dynamic heterogeneous redundancy based router architecture with mimic defenses[J]. Journal of Cyber Security, 2017, 2(1): 29-42.)
- [26] 郭军利, 许明洋, 原浩宇, 等. 引入内生安全的零信任模型[J]. 郑州大学学报: 理学版, 2022, 54(6): 51-58. (Guo Junli, Xu Mingyang, Yuan Haoyu, et al. Introduction of endogenous security of zero trust model[J]. Journal of Zhengzhou University: Natural Science Edition, 2022, 54(6): 51-58.)
- [27] 高岩, 王丹阳, 冯四风, 等. 一种拟态身份认证网关防御方案[J]. 小型微型计算机系统, 2020, 41(9): 1905-1911. (Gao Yan, Wang Danyang, Feng Sifeng, et al. Defense scheme of mimic identity authentication gateway[J]. Journal of Chinese Computer System, 2020, 41(9): 1905-1911.)
- [28] 郭威, 谢光伟, 张帆, 等. 一种分布式存储系统拟态化架构设计与实现[J]. 计算机工程, 2020, 46(6): 12-19. (Guo Wei, Xie Guangwei, Zhang Fan, et al. Design and implementation of a mimic architecture for distributed storage system[J]. Computer Engineering, 2020, 46(6): 12-19.)
- [29] 武兆琪, 张帆, 郭威, 等. 一种基于执行体异构度的拟态裁决优化方法[J]. 计算机工程, 2020, 46(5): 12-18. (Wu Zhaoqi, Zhang Fan, Guo Wei, et al. A mimic arbitration optimization method based on heterogeneous degree of executors[J]. Computer Engineering, 2020, 46(5): 12-18.)
- [30] 李卫超, 张铮, 王立群, 等. 基于拟态防御架构的多冗余裁决建模与风险分析[J]. 信息安全学报, 2018, 3(5): 64-74. (Li Weichao, Zhang Zheng, Wang Liqun, et al. The modeling and risk assessment on redundancy adjudication of mimic defense[J]. Journal of Cyber Security, 2018, 3(5): 64-74.)
- [31] 扈红超, 陈福才, 王祺鹏. 拟态防御 DHR 模型若干问题探讨和性能评估[J]. 信息安全学报, 2016, 1(4): 40-51. (Hu Hongchao, Chen Fucui, Wang Zhenpeng. Performance evaluations on DHR for cyberspace mimic defense[J]. Journal of Cyber Security, 2016, 1(4): 40-51.)
- (上接第1193页)
- [10] Xiong Yan, Su Cheng, Huang Wenchao, et al. SmartVerif: push the limit of automation capability of verifying security protocols by dynamic strategies[C]//Proc of the 29th USENIX Conference on Security Symposium. 2020: 253-270.
- [11] Cremers C, Horvat M, Hoyland J, et al. A comprehensive symbolic analysis of TLS 1.3[C]//Proc of ACM SIGSAC Conference on Computer and Communications Security. New York: ACM Press, 2017: 1773-1788.
- [12] Stephens J, Ferles K, Mariano B, et al. SmartPulse: automated checking of temporal properties in smart contracts[C]//Proc of IEEE Symposium on Security and Privacy. Piscataway, NJ: IEEE Press, 2021: 555-571.
- [13] He Xudong, Liu Qin, Chen Shuang, et al. Analyzing security protocol Web implementations based on model extraction with applied Pi calculus[J]. IEEE Access, 2020, 8: 26623-26636.
- [14] 张协力, 祝跃飞, 顾纯祥, 等. C2P: 基于 Pi 演算的协议 C 代码形式化抽象方法和工具[J]. 软件学报, 2021, 32(6): 1581-1596. (Zhang Xieli, Zhu Yuefei, Gu Chunxiang, et al. C2P: formal abstraction method and tool for C protocol code based on Pi calculus[J]. Journal of Software, 2021, 32(6): 1581-1596.)
- [15] Aizatulin M, Gordon A D, Jyurjens J. Computational verification of C protocol implementations by symbolic execution[C]//Proc of ACM Conference on Computer and Communications Security. New York: ACM Press, 2012: 712-723.
- [16] Petrosyan G R, Ter-Vardanyan L A. Modelling of identification and secret-key generation system with colored Petri net[C]//Proc of International Conference on Control, Decision and Information Technologies. Piscataway, NJ: IEEE Press, 2016: 239-245.
- [17] 刘嘉勇, 韩家璇, 黄诚. 源代码漏洞静态分析技术[J]. 信息安全学报, 2022, 7(4): 100-113. (Liu Jiayong, Han Jiaxuan, Huang Cheng. Vulnerability detection in source code using static analysis[J]. Journal of Cyber Security, 2022, 7(4): 100-113.)